

Comparative Study of Chatbot Architectures Using LLMs and RAG: Evaluating FAISS-Based Knowledge Retrieval Schemes

1st Herlawati Herlawati

Informatics

Universitas Bhayangkara Jakarta Raya

Jakarta, Indonesia

herlawati@ubharajaya.ac.id

2nd Ben Rahman

Information Systems

Universitas Nasional

Jakarta, Indonesia

benrahman@civitas.unas.ac.id

3rd Rahmadya Trias Handayanto

Informatics

Universitas Islam 45

Bekasi, Indonesia

rahmadya.trias@gmail.com

4th Prima Dina Atika

Informatics

Universitas Bhayangkara Jakarta Raya

Jakarta, Indonesia

prima.dina@dsn.ubharajaya.ac.id

5th Dwipa Handayani

Informatics

Universitas Bhayangkara Jakarta Raya

Jakarta, Indonesia

dwipa.handayani@dsn.ubharajaya.ac.id

6th Maimunah Maimunah

Informatics Engineering

Universitas Muhammadiyah Magelang

Magelang, Indonesia

maimunah@unimma.ac.id

7th Endang Retnoningsih

Informatics Engineering

Institut Bisnis Muhammadiyah Bekasi

Bekasi, Indonesia

endangretno@ibm.ac.id

8th Syahbaniar Rofiah

Informatics Engineering

Institut Bisnis Muhammadiyah Bekasi

Bekasi, Indonesia

rsyahbaniar@gmail.com

9th Didik Setiyadi

Management

STIE Arlindo

Bekasi, Indonesia

ddk.setiyadi20@gmail.com

Abstract— The use of Large Language Models (LLMs) for chatbot applications is currently widespread. The availability of various models with specific characteristics tailored to different needs has made these models increasingly popular. However, challenges remain, particularly the high computational resource requirements, such as the need for a Graphics Processing Unit (GPU). This has led to the need for optimization through emerging technologies, one of which is Retrieval-Augmented Generation (RAG). RAG allows users to avoid resource-intensive fine-tuning processes. Several RAG methods are already in use, such as Facebook AI Similarity Search (FAISS), Weaviate, Elasticsearch, Qdrant, etc., but the implementation techniques vary widely and still require experimentation to achieve optimal performance while minimizing computational demands. This is especially important considering that chatbots are often integrated into web servers, which are typically focused on web programming rather than artificial intelligence (AI). This study evaluates several chatbot schemes utilizing RAG with a FAISS index for a Question and Answer (Q/A)-based chatbot. The model used is a lightweight one with minimal GPU requirements, namely Gemma:2b-Instruct. The tested schemes include: (i) a single FAISS index, (ii) a single FAISS index with special chunking, (iii) two FAISS indices handling Q/A and non-Q/A in a serial manner, and (iv) two FAISS indices handling Q/A and non-Q/A in parallel. Experimental results show that using special chunking to handle Q/A improves accuracy, and the use of two FAISS indices in parallel minimizes the dominance of the Q/A index over the non-Q/A index.

Keywords— LLMs, Chatbot, RAG, FAISS, Q/A System, Gemma:2b-Instruct.

I. INTRODUCTION

The emergence of Large Language Models (LLMs) has made certain applications, such as chatbots, feel increasingly human-like in interaction. Their capabilities in text generation, reasoning, and context understanding have enabled AI to take over tasks that were once exclusively performed by humans.

The arrival of open-access LLM providers—such as DeepSeek [1], Mistral [2], Meta’s LLaMA [3], Gemma [4], and Qwen [4]—allows users to implement AI directly on their own servers without requiring paid subscriptions. However, the main challenge lies in the substantial computational resources required to run current LLM engines—particularly in terms of memory and Graphics Processing Unit (GPU) demands.

Fine-tuning is currently the most widely used method, where a baseline model is adapted for more specific use cases. This approach, which falls under the category of transfer learning, requires substantial computational resources and, as shown in previous studies, the training process can sometimes take several days to complete [5]. In addition to the need for substantial computational resources, some chatbots require continuously updated information. Using fine-tuning in this context necessitates re-fine-tuning whenever new data arrives. For large datasets, this also requires effort for forgetting—to ensure outdated or irrelevant information is properly removed from the model [6].

The use of a vector database containing new information to answer chat queries offers a distinct advantage, as it eliminates the need for fine-tuning. One well-known method is Facebook AI Similarity Search (FAISS), which, according to surveys, has been widely utilized in numerous studies [7]. Several studies focus on optimizing RAG, for example through Low-Rank Adaptation [8], Max-Min Semantic Chunking of Documents for RAG applications [9], and HybridRAG, which combines RAG with Knowledge Graphs [10].

Several studies have investigated this aspect, particularly focusing on ranking schemes [11]. Previous studies have compared FAISS with Chroma, and the results showed that FAISS is often faster and more lightweight for pure similarity searches, while Chroma offers greater flexibility and metadata integration, making it suitable for more complex use cases [12].

Based on those previous studies involving the use of RAG in LLM implementations, this research aims to evaluate

several RAG schemes. We chose the fast FAISS system with several schemes for question-and-answer problems and information retrieval from specific documents in chatbot system. The tested schemes include: (i) a single FAISS index, (ii) a single FAISS index with special chunking, (iii) two FAISS indices handling Q/A and non-Q/A in a serial manner, and (iv) two FAISS indices handling Q/A and non-Q/A in parallel. Gemma:2b-Instruct is used as the LLM due to its characteristics such as being lightweight, open-source, instruction-tuned, and capable of running efficiently on minimal GPU resources.

The remainder of this paper discusses the tested RAG schemes, followed by performance evaluation using sample data for the FAISS index. After the discussion, the paper concludes with a summary of findings.

II. RESEARCH METHOD

The application of RAG (Retrieval-Augmented Generation) in chatbots offers several advantages over fine-tuning, such as: faster deployment, no need for retraining, and the ability to update knowledge dynamically. Several RAG scenarios utilizing FAISS indexing are analyzed and their performance is evaluated.

A. Datasets

RAG uses ‘LangChain,’ a Python library for building modular and extensible applications powered by Large Language Models (LLMs). It provides tools to integrate LLMs with external data sources, such as documents, databases, or APIs, enabling the models to retrieve relevant information and generate more accurate and context-aware responses. Conversion from PDF files is necessary to match the vector embeddings used in RAG. The format used is question and answer, with “Q:” indicating a question and “A:” indicating an answer. The second type of data consists of PDFs in the form of book manuscripts.

Another dataset consists of academic papers in PDF format, which must first be converted into vector representations. Unlike the Q/A dataset, which clearly defines question and answer boundaries, academic papers lack such structure. As a result, text chunks are created based solely on length—for example, 300 or 400 tokens per chunk. Some methods apply overlapping by including a few words from the previous chunk to preserve context and ensure continuity between chunks.

B. Facebook AI Similarity Search (FAISS)

FAISS is a fast similarity search technique developed by Facebook AI Research. This technique supports Retrieval-Augmented Generation (RAG) by enabling efficient and scalable retrieval of relevant documents or data chunks from large vector databases, thereby enhancing the quality and relevance of generated responses.

Figure 1 illustrates how FAISS works when a prompt requests specific information from the vector database. The LLM, in this case Gemma, then responds based on the embedded vector representation of the prompt.

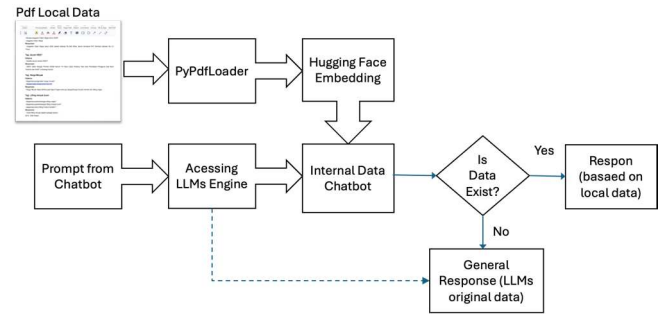


Fig. 1. RAG chatbot architecture using FAISS index

The FAISS index stores vector data in small chunks that can be quickly retrieved based on the similarity of the prompt entered into the chatbot. These retrieved chunks, referred to as context, are then sent to the LLMs to generate a response according to the given instructions. The result is then returned to the user. Sometimes, the search may not find relevant information within the context (FAISS index). If no relevant context is found, the LLMs will either respond based on their own knowledge or indicate that they do not know the answer. Some vendors provide FAISS-compatible services or tools, such as Hugging Face, Zilliz, and Milvus.

C. Gemma:2b-Instruct

Many LLM models are now widely available, with most of them being free to use, such as GPT, LLaMA, PaLM, DeepSeek [1] and Gemma. Since this study focuses on vector data management, for fairness, each scenario uses the same LLM model, namely gemma:2b-instruct. This model was developed by Google DeepMind as part of the Gemma family, designed for efficient instruction-following tasks. It is an open, lightweight model suitable for various natural language understanding and generation tasks, and can be deployed locally with relatively low resource requirements.

Gemma was installed via Ollama (<https://ollama.com>), a platform that hosts a wide range of state-of-the-art large language models (LLMs), such as LLaMA 2, LLaMA 3, Gemma, Mistral, and Phi. There are several versions of Gemma, including Gemma:2B, Gemma:2B-Instruct, Gemma:7B, and Gemma:7B-Instruct. Each version has its own strengths and weaknesses in terms of accuracy and GPU requirements. For instance, the 2B variants are more lightweight and suitable for deployment on consumer-grade hardware, while the 7B variants offer higher accuracy but require more powerful GPUs for optimal performance.

D. Scenarios

There are four scenarios that will be examined in this study. The first scenario directly utilizes indexing with chunking based on character count, applied to both question/answer (Q/A) types and documents (books, reports, and so on). The pseudocode below illustrates how the second scenario utilizes regular expressions to capture questions and answers within a single chunk. The chunk length adapts to the length of each question and answer pair. When creating a Q/A context, users need to format the data according to the Q: and A: template so that FAISS can properly interpret and structure the chunks.

```

pattern = r"(Q\s*:\s*.*?)(A\s*:\s*.*?)(?=\nQ\s*:\s*|\nZ)"
matches = re.findall(pattern, merged_text, flags=re.DOTALL)
chunks = []
for i, (q, a) in enumerate(matches):
    qa_text = f"{q.strip()}\n{a.strip()}"
    metadata = {
        "doc_id": "data_qa",
        "chunk_index": i,
        "source": pdf_path
    }
    chunks.append(Document(page_content=qa_text,
                           metadata=metadata))

```

The second scenario uses regular expressions, specifically for Q/A types, where each chunk is split per individual question. The third scenario separates the two types of RAG data in a serial process: Q/A is queried first, and if no match is found (below a certain threshold), the search continues in the vector data of the documents (Fig 2).

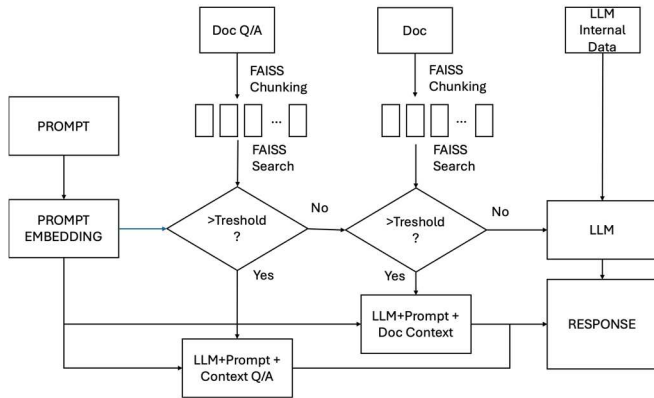


Fig. 2. Two Serial FAISS Indexes Scheme

The final scenario performs a parallel search across both Q/A and document sources (Fig 3). The performance of these four scenarios will be compared based on the responses generated from the RAG vector data. The RAG workflow operates such that when a prompt is received, the chatbot searches the available RAG data; if no relevant data is found, the chatbot responds using its own knowledge (for general information) or states “unknown” if the information is not found in the RAG.

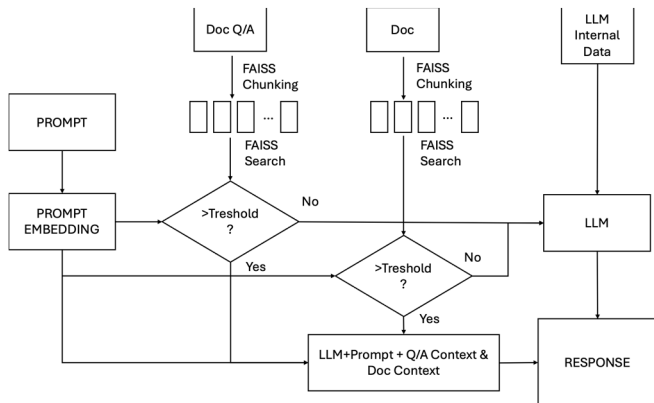


Fig. 3. Two Parallel FAISS Indexes Scheme

This study uses an M3 processor on a MacBook Air with a GPU featuring a 10-core architecture (Apple M3 integrated GPU). Accuracy is measured by comparing the chatbot’s

responses with the RAG vector data (both Q/A and document-based).

III. RESULT AND DISCUSSION

For the experiment, two datasets were used: a Q/A dataset and a document dataset. These two datasets have different characteristics. While the Q/A dataset consists of questions and answers, the document dataset contains information structured in chapters, subchapters, and paragraphs. To evaluate the performance of FAISS in retrieving information based on prompt questions, a direct (human) evaluation was conducted on all questions from the Q/A dataset and on a sample of questions from the document dataset.

A. Speed Performance

The response speed of LLMs to prompts depends on the Graphic Processing Unit (GPU). Since the experiments were conducted on the same device using the same LLM model (Gemma:2b-Instruct), the processing speed did not vary significantly between one scenario and another, ranging from approximately 15 to 20 seconds.

The processing time here primarily refers to the LLM’s response to a prompt. The FAISS retrieval speed itself is under one second. The response time to a prompt depends on the number of chunks selected during the FAISS indexing process. Larger chunks generally result in longer response times. The complexity of the task given to the LLM also affects the duration, such as whether it involves summarizing, calculating, or performing other functions.

B. Accuracy

Table 1 shows the accuracy comparison between FAISS indexing with and without regular expressions. The use of regular expressions ensures that each chunk remains within the boundaries of a question (Q:) and its corresponding answer (A:). In contrast, standard FAISS indexing relies on fixed chunk lengths (in characters), which can result in chunks being split in the middle of a Q: and A: pair. Although there are additional mechanisms besides chunking — such as context window overlap and semantic similarity matching — to mitigate information loss due to splitting, lightweight LLMs like Gemma:2b-Instruct still struggle to handle such cases, especially when the prompt involves complex tasks such as summarization, translation, calculation, generating charts, and similar operations.

TABLE I. TESTING RESULT OF SCENARIO 1 AND SCENARIO 2

		Accuracy (%)	Speed Average (Sec)
Scenario			
1.	Basic FAISS	72	5-10
2.	Regular Expression FAISS	100	5-10

Scenario 3 requires careful consideration in setting the threshold. If the threshold is too strict, responses will tend to rely on the Q/A data, since it is checked first. This model is suitable when the chatbot's main focus is delivering information from the Q/A dataset. However, if the Q/A content is too general, some information that is more thoroughly covered in the non-Q/A context (document context) may not be retrieved, as the Q/A match is already found first. Certain tasks that are not related to Q/A—such as generating presentations that depend on document context—cannot be handled easily. Figure 4 shows the results of Scenario 3.



Fig. 4. Scenario 3

Figure 4 shows the chatbot responding based on the Q/A context with a similarity score of 0.878. Since this exceeds the threshold during the Q/A context retrieval, the chatbot provides an answer based on that index and does not proceed to the document context. This scenario is highly suitable for systems that require precise and factual information, such as registration schedules, procedures/SOPs, specific links, and similar items. However, for complex prompts that require information from the document context, if the Q/A similarity exceeds the threshold, the document context will be skipped, resulting in reduced accuracy in the chatbot's response.

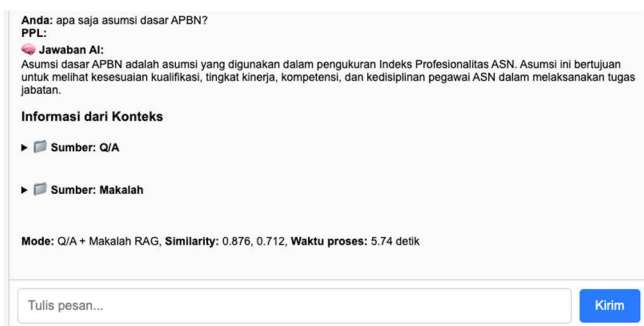


Fig. 5. Scenario 4

Figure 5 illustrates Scenario 4, in which FAISS performs parallel retrieval from both the Q/A context and the document context to supply relevant information to the LLM. The context from both Q/A and Document sources is retrieved

simultaneously. To simplify the chat interface, the context is hidden by default and can be revealed when the user clicks the 'expand' symbol to the left of the context source (Q/A and Document).

The chunks retrieved from the Q/A and document contexts are then passed to the LLM to generate a response based on the given prompt. It is important to note that the LLM is capable of handling texts in various languages, including Indonesian. If no relevant information is found in either the Q/A or document contexts, the chatbot will instruct the LLM to answer based on its general knowledge. If that is not possible, it will respond with a predefined fallback answer based on a pre-prompt — a default prompt embedded in the program code — when no relevant data is retrieved from the RAG (FAISS index), as shown in Figure 6.



Fig. 6. The use of internal data by LLMs

The chatbot could not find information about who the president of Russia is within the provided context (both Q/A and document) and thus responded based on its internal training data. For Gemma:2b-Instruct, the latest training data is up to 2023. If asked for information beyond that year, the LLM cannot provide an accurate answer. Figure 7 shows the code snippet as the initial prompt where the chatbot is instructed to respond using the appropriate language and style when no answer can be retrieved from the context (Q/A or document).

```
class LocalLLM(LLM):
    url: ClassVar[str] = "http://127.0.0.1:11434/api/generate"
    _executor: ClassVar[concurrent.futures.ThreadPoolExecutor] = concurrent.futures.ThreadPoolExecutor(max_workers=4)

    def _call(self, prompt: str, stop: Optional[List[str]] = None) -> str:
        payload = {
            "model": "gemma:2b-instruct",
            "#model": "llama3",
            "prompt": prompt.strip(),
            "stream": False,
            "options": {"temperature": 0.5},
            "system": "Jawab dengan Bahasa Indonesia. Jika tidak tersedia konteks, gunakan pengetahuan Anda sendiri."
        }
```

Fig. 7. The use of the initial prompt when no context information is provided

Experiments were conducted using LLMs installed locally on a computer. A GPU with large VRAM is required to handle prompt questions along with the context retrieved from the FAISS index. Some LLMs have limitations when processing complex questions. To address this, the chatbot can implement commercial LLMs accessible via Application Programming Interfaces (APIs), such as ChatGPT, which has now entered version 5. The capabilities of generating images, graphs, and multimedia are increasingly powerful. Some LLMs, although available for free installation on local machines—such as DeepSeek—also provide paid API access for users without the budget to host large GPU servers. This is important considering that the full version of DeepSeek requires high-end GPUs: for example, DeepSeek-VL 7B needs at least an NVIDIA A100 40GB or RTX 4090 24GB for inference, while DeepSeek-VL 67B requires multiple A100 80GB GPUs or an H100 cluster for proper performance.

Figure 8 shows the use of paid APIs, e.g., Gemini, to reduce the load on servers typically used for standard web applications. Another method is to provide a dedicated server for the chatbot and supply the chatbot-hosting web server via an API, thereby keeping the web server lightweight. Web servers generally rely on cloud hosting, which often does not provide GPU capabilities.

Several LLM API providers offer free access, making it easier for developers to utilize these models. Once the application is running, users can actively subscribe to paid plans. Experiments show that gemini-1.5-flash is capable of handling the presentation creation process based on specific instructions, with the output saved as a PPTX file (Fig. 8 – below).

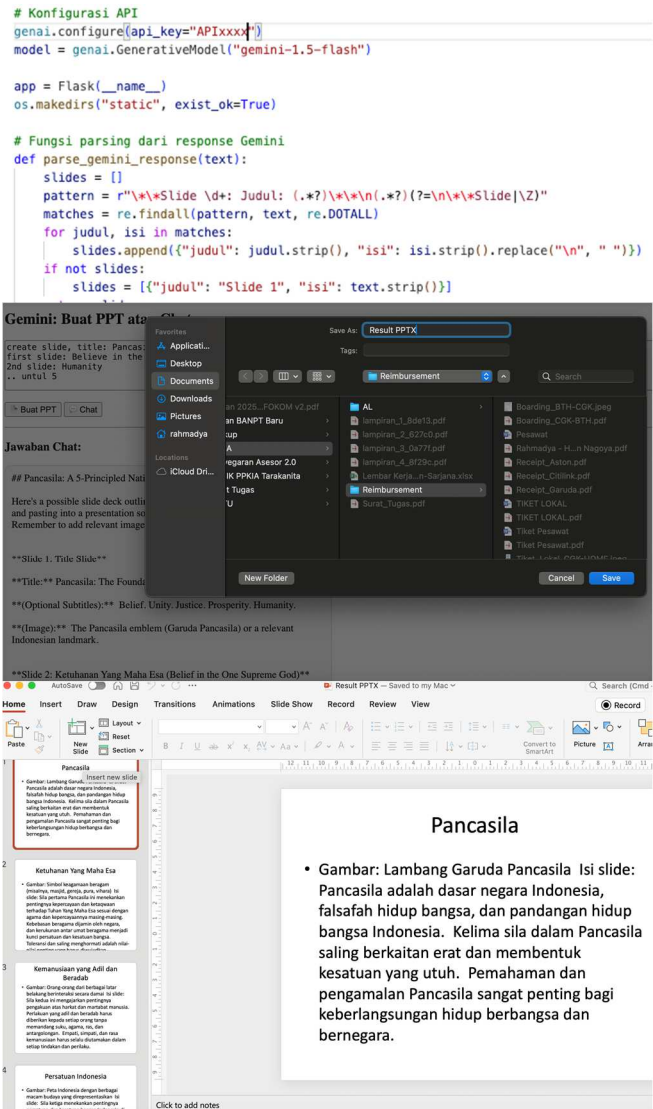


Fig. 8. Use of paid LLM APIs, e.g., Gemini (above), for Slide Generation Result (Below)

Table 2 shows a comparison between Scenario 3 (serial) and Scenario 4 (parallel). Each has its own advantages depending on the needs—whether for simple questions or more demanding NLP tasks such as summarization, essay generation, calculations, and others.

Testing Result Scenario 4 requires a longer processing time for the same LLMs (model and hardware

implementation), but it has the advantage of covering the entire RAG data (Q/A and documents). In the process of creating PowerPoint (PPT) presentations, Scenario 4 performs well, as generating presentations based on RAG context requires a large number of chunks and characters to produce complete and accurate results.

As shown in Figure 8, using the Gemini API allows the PPTX creation process to be completed quickly. However, using a locally installed model requires a longer processing time, since most server hardware (GPU and CPU) is not designed for LLMs but rather for standard web server operations. The generation of PPTX files is part of an active research area known as action chatbots, where the chatbot performs specific actions based on user commands.

TABLE II. TESTING RESULT OF SCENARIO 3 AND SCENARIO 4

Aspect	Scenario 3: Q/A Focused	Scenario 4: Q/A and Document Focused
Main Focus	Q/A context only	Both Q/A and Document context
Threshold Dependency	Highly dependent on similarity threshold	Threshold is less dominant
Response Speed	Faster, but may get trapped in Q/A context	Slower, requires stronger LLMs
Use Case Fit	Suitable when Q/A is the primary source	Suitable when both Q/A and documents are equally important
LLMs Requirement	Works with lightweight local LLMs	Requires more capable or commercial LLMs
Resource Requirements	Suitable for environments with limited GPU resources	Suitable for high-resource environments or paid API integration

For small servers, even Q/A-based context retrieval can sometimes take a long time to process. If the query involves short information, e.g., phone numbers, addresses, etc., a lookup link table technique can be used to speed up the response. Figure 9 shows that this method achieves very fast performance (only 0.01 seconds), compared to direct LLM processing (see the third question in Figure 9), which takes around 3 seconds. However, users need to upload a CSV file containing names and their corresponding links. Using fuzzy similarity is helpful so that queries do not have to match the CSV link table exactly—for example, “*tek informasi*” can still be matched with “*teknologi informasi*.”

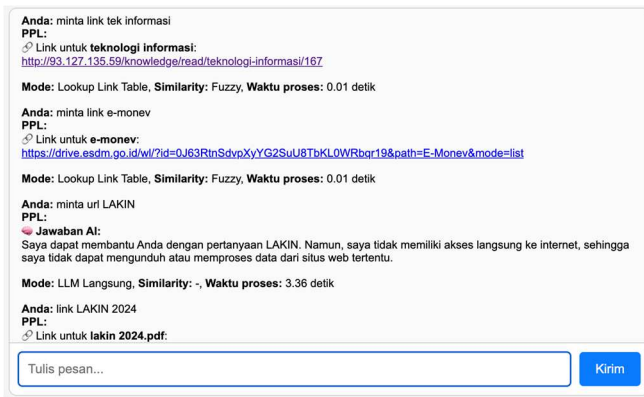


Fig. 9. The use of a lookup table with fuzzy similarity matching

In addition to understanding the libraries required for building a chatbot, knowledge of JavaScript is essential, as practical applications often demand specific capabilities—such as clickable links that direct users to the appropriate URLs, or converting paragraph-based information into structured tables. Several chatbot platforms, such as ChatGPT, also offer generative features like creating charts from existing tables, generating images from prompts, and other similar functionalities that are rapidly evolving.

IV. CONCLUSION

The application of LLMs in chatbots remains an active area of research, especially as new state-of-the-art models continue to emerge—improving both in terms of information coverage and specialized capabilities such as image generation, video creation, and advanced text composition. Experiments using lightweight LLMs, such as Gemma:2b-Instruct, demonstrate that with the right techniques, effective domain-specific chatbot applications can be developed. These can range from chatbots handling short, fact-based queries to those performing advanced NLP tasks like generating tables, visualizations, or retrieving direct links from websites. The experiments showed that dynamic chunking (Scenario 2), which segments context based on question–answer boundaries, performs well when the system relies solely on chunk length. Moreover, the parallel system in Scenario 4 outperformed the serial processing in Scenario 3, although it requires more powerful LLMs and hardware to handle large-context prompts effectively. This study was conducted in a zero-shot setting on a limited sample dataset. While this demonstrates the feasibility of lightweight LLMs for RAG-based chatbots, the results should not be overgeneralized. Future research should include larger and more diverse datasets, statistical validation with standardized metrics (e.g., F1-score, BLEU, ROUGE), and baseline comparisons with non-RAG approaches to ensure replicability and robustness. Further work on Action Chatbots is also especially promising, where the existing scenarios can be extended to build autonomous systems capable not only of providing information but also executing tasks. However, such capabilities necessitate ethical oversight, as autonomous

chatbot systems with action-taking abilities could pose significant risks if left unchecked.

ACKNOWLEDGMENT

The authors thank to Universitas Bhayangkara Jakarta Raya, Universitas Nasional, Universitas Islam 45, Universitas Muhammadiyah Magelang, Institut Bisnis Muhammadiyah Bekasi, and STIE Arlindo for supporting the research. The authors also thank to reviewer for insightful comments.

REFERENCES

- [1] L. Xiong *et al.*, “DeepSeek: Paradigm Shifts and Technical Evolution in Large AI Models,” *IEEE/CAA J. Autom. Sin.*, vol. 12, no. 5, pp. 841–858, 2025, doi: 10.1109/JAS.2025.125495.
- [2] N. Abbas and E. Atwell, “Cognitive Computing with Large Language Models for Student Assessment Feedback,” *Big Data Cogn. Comput.*, vol. 9, no. 5, pp. 1–21, 2025, doi: 10.3390/bdcc9050112.
- [3] H. Touvron *et al.*, “Llama 2: Open Foundation and Fine-Tuned Chat Models,” 2023, [Online]. Available:
- [4] T. Nguyen, L.-V. Hoang, and H.-D. Tran, “Qwen vs. Gemma Integration with Whisper: A Comparative Study in Multilingual SpeechLLM Systems,” pp. 1–4, 2025, [Online]. Available: <http://arxiv.org/abs/2506.13596>
- [5] M. Mathebula, A. Modupe, and V. Marivate, “Fine-Tuning Retrieval-Augmented Generation with an Auto-Regressive Language Model for Sentiment Analysis in Financial Reviews,” *Appl. Sci.*, vol. 14, no. 23, pp. 1–43, 2024, doi: 10.3390/app142310782.
- [6] H. Li, L. Ding, M. Fang, and D. Tao, “Revisiting Catastrophic Forgetting in Large Language Model Tuning,” *EMNLP 2024 - 2024 Conf. Empir. Methods Nat. Lang. Process. Find. EMNLP 2024*, pp. 4297–4308, 2024, doi: 10.18653/v1/2024.findings-emnlp.249.
- [7] A. Salemi and H. Zamani, “Evaluating Retrieval Quality in Retrieval-Augmented Generation,” *SIGIR 2024 - Proc. 47th Int. ACM SIGIR Conf. Res. Dev. Inf. Retr.*, pp. 2395–2400, 2024, doi: 10.1145/3626772.3657957.
- [8] Y. Choi, S. Kim, Y. Cedric, and F. Bassole, “Enhanced Retrieval-Augmented Generation Using Low-Rank Adaptation,” pp. 1–19, 2025.
- [9] C. Kiss, M. Nagy, and P. Szilágyi, “Max–Min semantic chunking of documents for RAG application,” *Discov. Comput.*, vol. 28, no. 1, 2025, doi: 10.1007/s10791-025-09638-7.
- [10] B. Sarmah, D. Mehta, B. Hall, R. Rao, S. Patel, and S. Pasquali, “HybridRAG: Integrating Knowledge Graphs and Vector Retrieval Augmented Generation for Efficient Information Extraction,” in *Proceedings of the 5th ACM International Conference on AI in Finance*, in ICAIF ’24, New York, NY, USA: Association for Computing Machinery, 2024, pp. 608–616. doi: 10.1145/3677052.3698671.
- [11] N. A. Akbar, “Are Re-Ranking in Retrieval-Augmented Generation Methods Impactful for Small Agriculture QA Datasets? A Small Experiment,” *BIO Web Conf.*, vol. 167, 2025, doi: 10.1051/bioconf/202516701001.
- [12] T. Şakar and H. Emekci, “Maximizing RAG efficiency: A comparative analysis of RAG methods,” *Nat. Lang. Process.*, vol. 31, no. 1, pp. 1–25, 2025, doi: 10.1017/nlp.2024.53.
- [13] F. Koto *et al.*, “IndoBERTweet: A Pretrained Language Model for Indonesian Twitter with Effective Domain-Specific Vocabulary Initialization,” *arXiv*, 2021, doi: 10.48550/arXiv.2109.04607.